Nonlinear Mechanics and the Lorenz Attractor

Based on Chapter 12 of John R. Taylor's *Classical Mechanics*

May 4, 2020

1 Introduction and Background to Nonlinearity and Chaos

For a large number of systems that occur in both natural and industrial contexts, *chaos* plays an important role in describing the underlying mechanics at work and as such, understanding the physics that govern it is of great importance. Chaos itself is an inherently multidisciplinary concept, showing up a wide variety of fields that include the study of oscillating mechanical systems, chemical reactions, fluid flow, lasers, population growth, the spread of diseases, and many others. The theoretical, experimental, and mathematical study of chaos within physics is known as *Chaos Theory*.

As a whole, chaos is commonly defined as a property of complex, dynamical systems where a particular system, though governed by deterministic equations, may display random or seemingly unpredictable future behaviors that never repeat themselves exactly. One of the most important characteristics of chaotic mechanics is a system's *sensitivity to initial conditions*. This phenomenon tells us that two identical systems that vary ever so slightly in their initial conditions can evolve quite differently in time and exhibit varied behavior, especially at large time scales. This characteristically high sensitivity will be a property that we will exploit throughout our later analysis in Section 3 to help us determine when a particular system is exhibiting signs of chaos.

For chaos to exist, a system must be complex and governed by *nonlinear* equations of motion. A nonlinear differential equation is an equation where at least one of its dependent variables is nonlinear (i.e. expressed as a product of itself, expressed within a sine or cosine, etcetera). Two common examples of nonlinear equations include the equation of motion of a simple pendulum of mass m and length L

$$mL^2\ddot{\phi} = -mgL\sin(\phi) \tag{1}$$

and the equation of motion of a single mass m experiencing a gravitational force a distance r away from another mass M

$$m\ddot{\mathbf{r}} = -GM\hat{\mathbf{r}}/r^2, \tag{2}$$

where G is the gravitational constant. In both of these equations, the dependent variables ϕ and r are nonlinear due to the $\sin(\phi)$ and $\hat{r}/r^2$ terms respectively. A characteristic quality of nonlinear equations is that they are notoriously difficult or impossible to solve analytically due to their complexity, and the same applies for most chaotic systems that are governed by nonlinear equations. Within the analysis of Section 3, we will take advantage of such numerical methods to manipulate and solve for particular nonlinear differential equations to a specific chaotic system.

In Chaos Theory, a notable distinction to make between chaotic and nonlinear systems is that a system can be nonlinear but not chaotic, but all chaotic systems must be nonlinear. Thus, nonlinearity does not guarantee chaos, as can be seen in the example of the simple pendulum of Equation 1 which does not exhibit chaos even at large amplitudes.

Returning to previous discussions, an illustrative, metaphorical example of nonlinear, chaotic systems and their high sensitivity is the idea known as the "butterfly effect." This line of thought reasons that

a small change in one state of a chaotic system, such as a butterfly flapping its wings in one part of the world, can result in large, dramatic changes in another state, such as a tornado occurring on the other side of the world a week later. Taking this a step further, the "butterfly effect" argues that if these minor changes or disturbances within the local system had not occurred, the effects would not have occurred either.

The phrase "butterfly effect" was first coined by meteorologist Edward Lorenz, who noticed that any small scale changes to the starting point of his computer weather models resulted in anything from a violent rain storm to sunny skies (with no way to predict at the time what the final outcome would be). Lorenz's observations matched symptoms of chaos first noted by French mathematician Poincaré in his studies of three-body gravitational problems in the late 19th Century, pointing to a deeper picture of this complex physical phenomenon. Yet, due to the complexity of analyzing nonlinear equations without the aid of computers, it wasn't until advancements in computing and numerical simulations in the 1970s that the study of chaos really began to take shape and draw the attention of a significant number of physicists. Today, the study of chaos theory has continued to mirror the development of computers, resulting in significant traction within physics today due to a variety of recent discoveries.

2 The Lorenz System of Equations

The same Lorenz of Section 1 who coined the term "butterfly effect" is also responsible for a famous system of equations known as the *Lorenz System*. These equations were first conceived to describe a simplistic model of atmospheric convection, which occurs in nature when fluids in the air are heated uniformly from below and cooled uniformly from above in circulation pattern known as Rayleigh-Bénard convection. Developments in similar fields have resulted in similar equations for models in lasers, motors, circuits, and chemical reactions. As such, analyzing the Lorenz System has a plethora of applications in these similar fields and their similar equations.

The Lorenz System can be modeled by the following system of equations:

$$\begin{aligned} \dot{x} &= \sigma(y - x) \\ \dot{y} &= x(\rho - z) - y \\ \dot{z} &= xy - \beta z, \end{aligned} \tag{3}$$

where σ, ρ, and β are specific system parameters of the system that is being modeled or studied. The system of equations of Equation 3 contains three coupled equations that are nonlinear for the dependent variables x, y, and z, resulting in complex motion and a potential for chaotic behavior. In particular, the *Lorenz Attractor* is a set of chaotic solutions of the Lorenz System, and determining the conditions for these solutions will be the main focus of our following analysis. It is worth noting that these equations are defined for an arbitrary x, y, and z and as such, the discussion will be kept general and any solutions within analysis will be kept unitless so they can easily be applied to any applicable system.

3 Analysis of Parameters and Conclusions

In order to better understand what effects the parameters σ, ρ, and β have on the system of equations in Equation 3, we will perform analysis under specific sets of conditions. In our specific analysis, we will keep σ and β constant at values of $\sigma = 28$ and $\beta = 4$ and vary ρ to observe how it effects equilibrium positions, their stabilities, and the overall trajectories of various points. In doing so, we hope to determine at what ρ values we would get chaotic solutions corresponding to the Lorenz Attractor.

Before getting into our specific analysis, it is worth presenting a brief review/summary of the specific aspects we will look at in our analysis. To begin, equilibrium positions of a system are the points at which the system won't change, generally corresponding to when the first derivatives of dependent variables are equal to zero. Every equilibrium position has a corresponding stability associated with it that describes

whether that point is attractive from surrounding points (stable) or repulsive (unstable). Typically, a stable point has a positive second derivative while a unstable point has a negative second derivative. Due to the complexity of our specific equation (especially with further derivatives and algebra), we will approach discovering stability differently through plots of trajectories of surrounding points as will be seen later in Figures 1, 2, and 3.

Within our system of equations given by Equation 3, equilibrium points occur when $\dot{x} = \dot{y} = \dot{z} = 0$. After making the assumption that σ and β are always positive (which holds in our analysis since $\sigma = 28$ and $\beta = 4$), we can plug in the above result and simplify to derive a series of equilibrium relationships such that

$$\begin{aligned} y_i &= x_i \\ y_i &= x_i(\rho - z_i) \\ x_i y_i &= \beta z_i, \end{aligned} \tag{4}$$

must be true for a specific (x_i, y_i, z_i) point to be an equilibrium position for the Lorenz System. An obvious solution to these above relationships is at point (0, 0, 0), which corresponds to our first equilibrium point that is present regardless of what the parameter values are.

To find more equilibrium positions, the above relationships in Equation 4 can be further simplified by comparing each to each other to realize that $(\rho - z_i) = 1$ and $x_i^2 = y_i^2 = \beta z_i$. After some simple algabraic manipulation, we find that the positions

$$\begin{aligned} x_i &= \pm\sqrt{\beta(\rho - 1)} \\ y_i &= \pm\sqrt{\beta(\rho - 1)} \\ z_i &= \rho - 1, \end{aligned} \tag{5}$$

decode the final two possible equilibrium points of our specific system at the point(s) (x_i, y_i, z_i).

Now, we can observe the presence of these equilibrium points and their relative stabilities when $\sigma = 28$ and $\beta = 4$ and ρ is varied as first proposed. Beginning at $\rho = 0.5$, we already know the given equilibrium position of (0, 0, 0). Yet, when attempting to use the relationships given in Equation 5, we get a x_i and a y_i that are imaginary due to the fact that $\rho < 1$ and $\beta > 0$. As such, our only equilibrium position is at the point (0, 0, 0). To determine the stability of this point, we can refer to Figure 1 that plots the trajectory of a point that starts at (0.5, 0.5, 0.5) and has motion defined by the Lorenz System of Equation 3 over 20 seconds. This plot, as well as all future plots, was generated by code that can be found in Section 4.

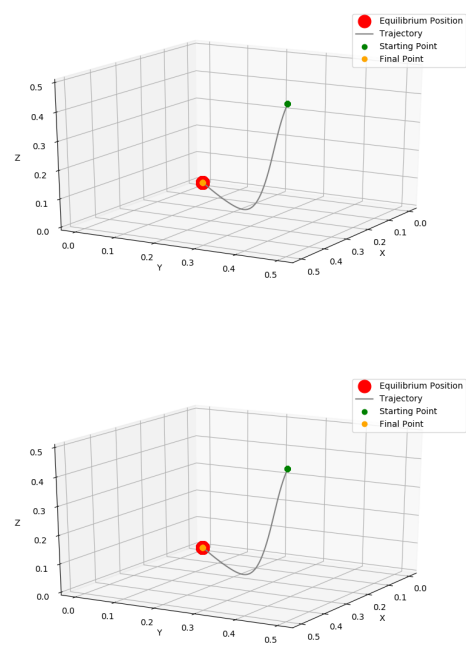

Figure 1: Trajectory of a point beginning near the equilibrium position (0, 0, 0) in the Lorenz System when $\rho = 0.5$. Both plots represent the same trajectory that is presented from two different viewing angles.

As can be seen in the above figure, our initial point is attracted to the equilibrium position (0, 0, 0) over the given time scale and we can conclude that this position is a stable one.

Next, increasing our ρ value such that $\rho = 10$ gives us two new equilibrium positions that can be derived by Equation 5. After plugging in given values for β

and σ into these new positions, this results in three total equilibrium positions of (0, 0, 0), (6, 6, 9), and (-6, -6, 9). As before, we can determine the stability of these positions through a plot that is given in Figure 2. This plot similarly shows the trajectory of two points of initial positions (2, 2, 2) and (-2, -2, -2) and motion defined by the Lorenz Equation over 20 seconds each.

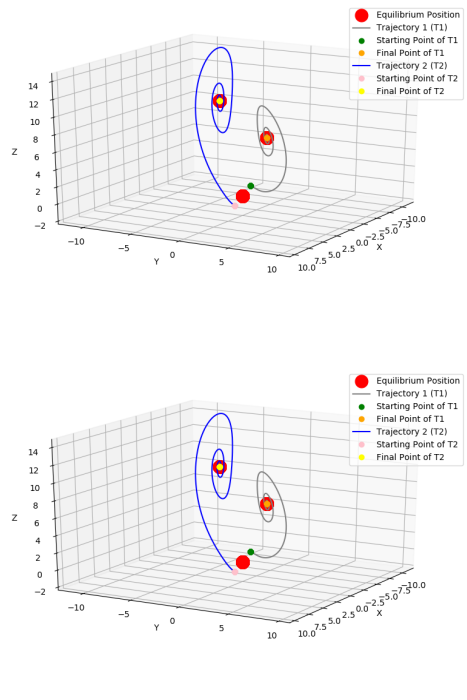

Figure 2: Trajectory of points beginning near equilibrium position (0, 0, 0) in Lorenz System when $\rho = 10$. Both plots represent the same trajectory that is presented from two different viewing angles.

As seen in this figure, though each trajectory begins near (0, 0, 0), they are each repulsed away from it and are instead attracted to points (6, 6, 9) and (-6, -6, 9) depending on which equilibrium position was closer. Thus, we can conclude that the equilibrium position (0, 0, 0) is now unstable at this value of ρ while the positions (6, 6, 9) and (-6, -6, 9) are stable instead.

Finally, setting $\rho = 46.92$ provides us with three equilibrium positions that are slightly shifted but still defined by the same relationships as before. Namely, by plugging in our given values for β and ρ into Equation 5 to get the position of our equilibrium points, we get equilibrium points of (0, 0, 0), (13.55, 13.55, 45.92), and (-13.55, -13.55, 45.92). We can then use the same process as before to determine stabilities of these positiosn by plotting a trajectory beginning at (5, 5, 5) over 20 seconds, as seen in Figure 3.

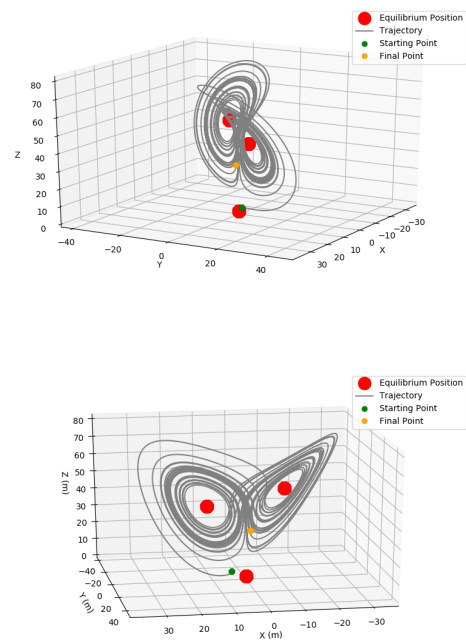

Figure 3: Trajectory of point beginning near equilibrium position (0, 0, 0) in Lorenz System when $\rho = 46.92$. Both plots represent the same trajectory that is presented from two different viewing angles.

We can observe in the above figure that the stability of our points has not changed, but the trajectory has taken on a shape that is characteristically chaotic, circling around the two equilibrium positions but never reaching them or repeating the same pattern of motion exactly. Once again, we can conclude

that the equilibrium position (0, 0, 0) is unstable, (13.55, 13.55, 45.92) is stable, and (-13.55, -13.55, 45.92) is stable at our specific value of ρ.

To verify that the behavior we observed in Figure 3 when $\rho = 46.92$ (and to check for similar signs of chaos in Figure 2 when $\rho = 10$), we can observe each specific system's sensitivity to initial conditions to determine whether they exhibit signs of chaos, as first alluded to in Section 1. We will accomplish this graphically once again, by plotting two trajectories over 20 seconds that initially start very close to each other but far away from any equilibrium point at (10, 2, 10) and (10.01, 2.01, 10.01) respectively. A comparison of how the motion of these two trajectories evolved in the Lorenz System can be seen in Figure 4.

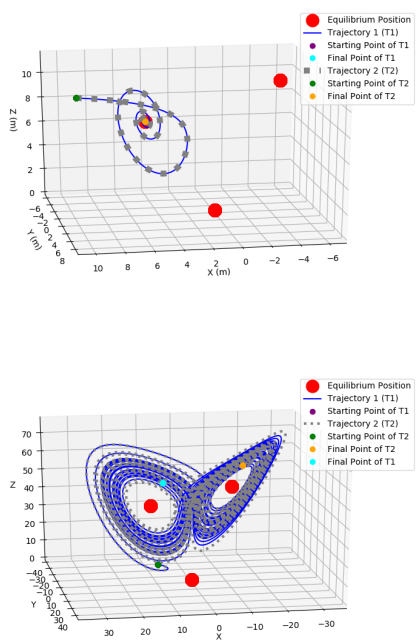

Figure 4: Comparison of the motion of two initially close points in Lorenz System when $\rho = 10$ (top) and $\rho = 46.92$ (bottom).

Qualitatively, it can be observed that the two trajectories of the top plot mirror each other closely, ending at the same point near the stable equilibrium point that they begin closest to. Yet, in the bottom plot, the two trajectories map out distinct paths from each other that appear to be of the same general shape but not the same specific motion. To put numbers to these plots, we calculated the distance between the final points of the two trajectories of both plots (as marked in both plots in Figure 4) using a typical distance formula whose code is given in Section 4. The results of these different distances, calculated from different Lorenz Systems at different values of ρ, are presented in Table 1.

ρ Value	Distance between Points
10	3.18×10^{-11}
46.92	28.22

Table 1: Comparison of distance between final positions of two trajectories of initial positions (10, 2, 10) and (10.01, 2.01, 10.01) that behave under the Lorenz System of equations. Distances were calculated using code from Section 4.

From these distances within the above table, we can conclude that at sufficiently large value of ρ (such as $\rho = 46.92$), the Lorenz System exhibits a sensitivity of initial conditions and thus exhibits signs of chaos due to the large distance between final points. Conversely, the system at $\rho = 10$ does not exhibit any signs of chaos due to the extremely small distance between final points. The chaotic solutions plotted in the trajectory of Figure 3 thus correspond to the Lorenz Attractor as first mentioned in Section 2.

We can now conclude this analysis and this paper by confirming the potential for chaotic behavior of the Lorenz System and by demonstrating that increasing the parameter ρ in this system of equations leads to signs of chaos at sufficiently high values.

4 Referenced Code

The code used to setup the creation of plots was as follows:

```
#Importing necessary packages

#Numpy for calculations and plotting
import numpy as np

#Matplotlib for plotting
import matplotlib.pyplot as plt
%matplotlib inline
plt.style.use('default')

#Odeint for solving equations of motions
from scipy.integrate import odeint

#Imported for 3D plotting
from mpl_toolkits.mplot3d import Axes3D

#Setup

#Defining given constants
sigma = 28 #given value of system parameter
beta = 4 #given value of system parameter
#rho is defined later based on its given value

#Defining given equations as function used within OdeInt
def DE(state, time):
    x, y, z = state #defining variables of state in DE
    derivs = [sigma*(y-x), x*(rho-z)-y, x*y-beta*z]#defining their derivatives that are returned below
    return derivs

#List of time values (same in Part A, B, and C)
tlist = np.linspace(0,20,10000) #10000 steps from 0 to 20 seconds

#Defining general ODeInt solution function
def sol(rho,x,y,z):
    return odeint(DE, [x, y, z], tlist)
```

The code used to plot and check stabilities of equilibrium points was of the following form. Note that this specific excerpt is taken from code used when $\rho = 10$.

```
#Solving for equations of motion using OdeInt
rho = 10 #given value of parameter for Part A
sols2 = sol(rho,2,2,2) #solution starting at point (2,2,2) w/ rho = 10
sols_new2 = sol(rho,-2,-2,-2) #solution starting at point (-2,-2,-2) w/ rho = 10

#Defining x, y, and z coordinates of second equilibrium position
ex = np.sqrt(beta*(rho-1))
ey = np.sqrt(beta*(rho-1))
ez = rho-1

#Plotting trajectory and points in 3D
```

```
fig = plt.figure(figsize = (10,6))
ax = fig.gca(projection='3d')
ax.plot(0*tlist, 0*tlist, 0*tlist, 'o', markersize = '15', color = 'red', label = 'Equilibrium Position') #plotting one equilibrium position
ax.plot(ex + 0*tlist, ey + 0*tlist, ez + 0*tlist, 'o', markersize = '15', color = 'red') #plotting second equilibrium position
ax.plot(-ex + 0*tlist, -ey + 0*tlist, ez + 0*tlist, 'o', markersize = '15', color = 'red') #plotting third equilibrium position

ax.plot(sols2[:, 0], sols2[:, 1], sols2[:, 2], color = 'gray', label = 'Trajectory 1 (T1)') #plotting trajectory using derived equations of motion
ax.plot(sols2[0, 0]+0*tlist, sols2[0, 1] + 0*tlist, sols2[0, 2] + 0*tlist, 'o', color = 'green', label = 'Starting Point of T1') #plotting initial point
ax.plot(sols2[9999, 0]+0*tlist, sols2[9999, 1] + 0*tlist, sols2[9999, 2] + 0*tlist, 'o', color = 'orange', label = 'Final Point of T1') #plotting approximate final point

ax.plot(sols_new2[:, 0], sols_new2[:, 1], sols_new2[:, 2], color = 'blue', label = 'Trajectory 2 (T2)') #plotting trajectory using derived equations of motion
ax.plot(sols_new2[0, 0]+0*tlist, sols_new2[0, 1] + 0*tlist, sols_new2[0, 2] + 0*tlist, 'o', color = 'pink', label = 'Starting Point of T2') #plotting initial point
ax.plot(sols_new2[9999, 0]+0*tlist, sols_new2[9999, 1] + 0*tlist, sols_new2[9999, 2] + 0*tlist, 'o', color = 'yellow', label = 'Final Point of T2') #plotting approximate final point

#Labeling axes of plot
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('Motion of Points Beginning Near Equilibrium Positions under Lorenz System of Equations When $rho=10$', pad=30)
ax.view_init(elev=20, azim=30) #used to rotate view around (elev is tilt and azim is swivel)

plt.legend() #creating legend

plt.show()
```

The code used to check sensitivity of initial conditions was of the following form. Note that this specific excerpt is taken from code used when $\rho = 46.92$.

```
#Displaying sensitivity of initial conditions

#Solving for equations of motion using OdeInt
rho = 46.92 #given value of parameter for Part A
sols6 = sol(rho,10,2,10) #solution starting at point (10,2,10) w/ rho = 10
sols7 = sol(rho,10.01,2.01,10.01) #solution starting at point (10.01,2.01,10.01) w/ rho = 10

#Plotting trajectory and points in 3D
fig = plt.figure(figsize = (10,6))
ax = fig.gca(projection='3d')
ax.plot(0*tlist, 0*tlist, 0*tlist, 'o', markersize = '15', color = 'red', label = 'Equilibrium Position') #plotting one equilibrium position
ax.plot(ex + 0*tlist, ey + 0*tlist, ez + 0*tlist, 'o', markersize = '15', color = 'red') #plotting second equilibrium position
```

```
    ax.plot(-ex + 0*tlist, -ey + 0*tlist, ez + 0*tlist, 'o', markersize = '15', color = 'red') #plotting third equilibrium position

    ax.plot(sols6[:, 0], sols6[:, 1], sols6[:, 2], color = 'blue', label = 'Trajectory 1 (T1)') #plotting trajectory using derived equations of motion
    ax.plot(sols6[0, 0]+0*tlist, sols6[0, 1] + 0*tlist, sols6[0, 2] + 0*tlist, 'o', color = 'purple', label = 'Starting Point of T1') #plotting initial point

    ax.plot(sols7[:, 0], sols7[:, 1], sols7[:, 2], ':', lw = 2, color = 'gray', label = 'Trajectory 2 (T2)') #plotting trajectory using derived equations of motion
    ax.plot(sols7[0, 0]+0*tlist, sols7[0, 1] + 0*tlist, sols7[0, 2] + 0*tlist, 'o', color = 'green', label = 'Starting Point of T2') #plotting initial point
    ax.plot(sols7[9999, 0]+0*tlist, sols7[9999, 1] + 0*tlist, sols7[9999, 2] + 0*tlist, 'o', color = 'orange', label = 'Final Point of T2') #plotting approximate final point

    ax.plot(sols6[9999, 0]+0*tlist, sols6[9999, 1] + 0*tlist, sols6[9999, 2] + 0*tlist, 'o', color = 'cyan', label = 'Final Point of T1') #plotting approximate final point
    #final ppoint plotted later so it stands out on plot

    #Labeling axes of plot
    ax.set_xlabel('X')
    ax.set_ylabel('Y')
    ax.set_zlabel('Z')
    ax.set_title('Motion of Two Initially Close Points under Lorenz System of Equations When $rho=46.92$', pad=30)
    ax.view_init(elev=20, azim=70) #used to rotate view around (elev is tilt and azim is swivel)

    plt.legend() #creating legend

    plt.show()
```

Finally, the code used to calculate distances between points was of the following form. Note that this specific excerpt is taken from code used when $\rho = 46.92$.

```
    #Calculating Distances between final points (using distance formula)

    d_partC = np.sqrt((sols6[9999,0]-sols7[9999,0])**2+(sols6[9999,1]-sols7[9999,1])**2+(sols6[9999,2]-sols7[9999,2])**2)
    print('Distance between final points of T1 and T2: ' + str(d_partC)) #displaying result
```